# Library Management System

## Backend API Reference

*For Frontend Integration Team*

Version 1.0 | Spring Boot 3.2.5 | Java 17

# 1. Architecture Overview

The LMS backend is a microservices system built with Spring Boot 3.2.5. It consists of four independently deployable services that communicate over HTTP via a Eureka service registry and load-balanced RestTemplate.

| Service | Port | Artifact ID | Responsibility |
|---|---|---|---|
| **LMS (Core)** | 8081 | lms | Books, Members, Transactions, Reservations |
| **Notification Service** | 8085 | notificationservice | Email dispatch via SendGrid |
| **Config Server** | 8888 | config-server | Centralised YAML config (Git-backed) |
| **Eureka Server** | 8761 | eureka-server | Service discovery & registry |

ℹ The frontend exclusively calls the LMS Core service on port 8081. All /lms/** endpoints are defined there.

## 1.1 Base URL

```
http://localhost:8081
```

All LMS API endpoints are prefixed with /lms/. E.g.: http://localhost:8081/lms/getAllBooks

## 1.2 Common Headers

| Header | Value | Notes |
|---|---|---|
| Content-Type | application/json | Required for POST / PUT requests with body |
| Accept | application/json | Recommended for all requests |

## 1.3 CORS Policy

CORS is enabled for the following origin. No additional setup is needed for local development.

```
Allowed Origin: http://localhost:5173
Allowed Methods: GET, POST, DELETE, PUT
Allowed Headers: *
```

## 1.4 Rate Limiting

All /lms/** endpoints are rate-limited per IP address.

| Profile | Max Requests | Window |
|---|---|---|
| dev | 100 / minute | 60 seconds |
| prod | 20 / minute | 60 seconds |

⚠ When the limit is exceeded the server returns HTTP 429 with body: {"error": "Rate limit exceeded. Try again later."}

## 1.5 Global Error Format

All unhandled exceptions return the following JSON structure:

```
{
  "message":   "<human-readable error>",
  "status":    500,
  "timestamp": "2024-01-15T10:30:00"
}
```

# 2. Domain Models & Enums

The following types are used across request and response payloads.

## 2.1 PlanType (enum)

| Value | Description |
|---|---|
| STANDARD | 5 USD/month. Borrowing limit: 10 books, 30 days, fine $0.25/day, max 2 reservations. |
| PREMIUM | 7 USD/month. Borrowing limit: 10 books, 30 days, fine $0.25/day, max 5 reservations. |

## 2.2 AddOns (enum)

| Value | Description |
|---|---|
| EXTENDED_BORROWING | Adds +2 to borrowing limit and +7 days to borrowing duration. Costs +$3/month. |

## 2.3 TransactionStatus (enum)

| Value | Description |
|---|---|
| ACTIVE | Book is currently borrowed by the member. |
| RETURNED | Book has been returned successfully. |
| FINE_PENDING | Book was returned overdue; fine has been calculated and is unpaid. |

## 2.4 ReservationStatus (enum)

| Value | Description |
|---|---|
| QUEUED | Member is in the waitlist; book is still borrowed by someone else. |
| ACTIVE | Book is now available; member has up to 2 days to pick it up. |
| FULFILLED | Member collected the reserved book (borrowing transaction created). |
| EXPIRED | Member did not collect within the 2-day window; reservation cancelled. |

## 2.5 NotificationType (enum)

| Value | Description |
|---|---|
| BORROWED | Sent when a book is successfully borrowed. |
| RETURNED | Sent when a book is returned on time. |

| Value | Description |
|---|---|
| **OVERDUE** | Sent when a book is returned late (fine applied). |
| **RESERVATION_CREATED** | Sent when a reservation is queued. |
| **RESERVATION_AVAILABLE** | Sent when a reserved book becomes available for pickup. |

# 3. Book Endpoints

Manage the library book catalogue. Books maintain an internal state machine: Available → Unavailable (0 copies) or Lost.

## 3.1 Endpoint Summary

| Method | Endpoint | Description |
|---|---|---|
| GET | /lms/getAllBooks | Retrieve all books in the catalogue |
| POST | /lms/addBook | Add a new book (supports undo) |
| POST | /lms/removeBook/{bookId} | Mark a book as unavailable (supports undo) |
| POST | /lms/undoLastBookOperation | Undo the most recent add or remove operation |
| GET | /lms/commandHistory | Get the list of recent book operations |
| DELETE | /lms/commandHistory | Clear the command history |

## 3.2 GET /lms/getAllBooks

Returns a list of all books including their current state and reservation/transaction lists.

### Request

No request body or parameters required.

### Response — 200 OK

```
[
  {
    "id":               -3456789012345,
    "isbn":             "978-0132350884",
    "title":            "Clean Code",
    "author":           "Robert Martin",
    "category":         "Technology",
    "publicationYear":  2008,
    "copiesAvailable":  5,
    "reservations":     [],
    "transactions":     [],
    "currentState":     { "stateName": "Available", "available": true }
  }
]
```

## 3.3 POST /lms/addBook

Adds a new book to the system. The operation is recorded in command history and can be undone.

### Request Body

| Field | Type | Required | Description |
|---|---|---|---|
| **isbn** | String | Yes | International Standard Book Number |
| **title** | String | Yes | Full title of the book |
| **author** | String | Yes | Author name(s) |
| **category** | String | Yes | Subject/genre category |
| **publicationYear** | Integer | Yes | 4-digit publication year |
| **copiesAvailable** | Integer | Yes | Number of physical copies to add (must be >= 1) |

### Example Request Body

```
{
  "isbn":            "978-0132350884",
  "title":           "Clean Code",
  "author":          "Robert Martin",
  "category":        "Technology",
  "publicationYear": 2008,
  "copiesAvailable": 5
}
```

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Book added successfully | "Book added with Id: -3456789012345" |
| **500** | Internal error | { "message": "...", "status": 500, "timestamp": "..." } |

## 3.4 POST /lms/removeBook/{bookId}

Marks a book as unavailable (Lost state, copies set to 0). Does not delete the book record. Supports undo.

### Path Parameter

| Field | Type | Required | Description |
|---|---|---|---|
| **bookId** | Long | Yes | The ID of the book to remove (obtained from getAllBooks or addBook response) |

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Book marked unavailable | "Book marked as unavailable: Clean Code" |
| **500** | Book not found / other error | { "message": "Book not found with ID: 123", "status": 500, ... } |

## 3.5 POST /lms/undoLastBookOperation

Reverses the most recent add or remove book command. History is maintained in a stack (max 50 entries).

### Request

No body or parameters.

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Undo successful | "Book addition undone: ID -345678 marked as unavailable" OR "Book 'Clean Code' restored with 5 copies available" |
| **200** | Nothing to undo | "Cannot undo: No commands to undo" |

## 3.6 GET /lms/commandHistory

Returns the current command history stack.

### Response — history exists

```
{
  "historySize":  3,
  "canUndo":      true,
  "lastCommand":  "Add Book: 'Clean Code' by Robert Martin (ISBN: 978-0132350884)",
  "allCommands": [
    "Add Book: 'Clean Code' by Robert Martin",
    "Remove Book: ID -345678 (The Pragmatic Programmer)",
    "Add Book: 'Refactoring' by Martin Fowler"
  ]
}
```

### Response — empty history

```
{
  "message":     "No commands in history",
  "historySize": 0,
  "canUndo":     false
}
```

# 4. Member Endpoints

Register and list library members. Each member is assigned a membership plan, an optional add-on, and membership duration dates are calculated automatically at registration time.

## 4.1 Endpoint Summary

| Method | Endpoint | Description |
|---|---|---|
| POST | /lms/registerMember | Register a new library member |
| GET | /lms/getAllMembers | Retrieve all registered members |

## 4.2 POST /lms/registerMember

### Request Body

| Field | Type | Required | Description |
|---|---|---|---|
| **userName** | String | Yes | Unique username for the member |
| **password** | String | Yes | Member account password |
| **emailId** | String | Yes | Email address — used for notifications |
| **phoneNumber** | String | Yes | Contact phone number |
| **planType** | PlanType | Yes | Membership plan. See Section 2.1. Values: STANDARD \| PREMIUM |
| **memberShipMonths** | Integer | Yes | Duration of membership in months (e.g. 12) |
| **addOns** | AddOns | No | Optional add-on. See Section 2.2. Values: EXTENDED_BORROWING |

### Example Request Body — Standard Plan

```
{
  "userName":         "john_doe",
  "password":         "secure123",
  "emailId":          "john@example.com",
  "phoneNumber":      "0871234567",
  "planType":         "STANDARD",
  "memberShipMonths": 12
}
```

### Example Request Body — Premium + Add-on

```
{
  "userName":         "jane_premium",
  "password":         "pass456",
  "emailId":          "jane@example.com",
  "phoneNumber":      "0879876543",
  "planType":         "PREMIUM",
```

```
  "memberShipMonths": 6,
  "addOns":           "EXTENDED_BORROWING"
}
```

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Member registered | "Member created with Id: -987654321\nSelected Plan of Member Standard Plan\nTotal Cost of the Plan 5" |
| **500** | Validation / server error | { "message": "...", "status": 500, ... } |

⚠ The response is a plain text string (not JSON). Parse the member ID from the first line if needed.

## 4.3 GET /lms/getAllMembers

Returns a list of all members with their plan, dates, reservations, and notifications. Sensitive data (transactions list) is hidden from the JSON output via @JsonIgnore.

### Response — 200 OK (abbreviated)

```
[
  {
    "id":                 -987654321,
    "userName":           "john_doe",
    "emailId":            "john@example.com",
    "phoneNumber":        "0871234567",
    "membershipStartDate": "2024-01-15",
    "membershipEndDate":   "2025-01-15",
    "membershipPlan": {
      "planName":              "Standard Plan",
      "cost":                  5,
      "maxReservationsAllowed": 2,
      "borrowingPolicy": {
        "borrowingLimit":         10,
        "fineRatePerDay":          0.25,
        "borrowingDurationInDays": 30
      }
    },
    "reservations":   [],
    "notifications":  []
  }
]
```

# 5. Transaction Endpoints

Handle book borrowing and returns. Transactions trigger email notifications automatically. A circuit breaker (Resilience4j) guards both endpoints — if the service fails repeatedly, a fallback message is returned instead of an exception.

## 5.1 Endpoint Summary

| Method | Endpoint | Description |
|---|---|---|
| GET | /lms/borrowBook/ | Borrow a book for a member |
| GET | /lms/processReturn/{transactionId} | Return a previously borrowed book |
| GET | /lms/getAllTransactions | Retrieve all transaction records |

⚠ borrowBook and processReturn use HTTP GET for historical reasons. No request body is needed — all parameters are passed as query params or path variables.

## 5.2 GET /lms/borrowBook/

Borrow a book for a member. Validates availability, member borrowing limit, and reservation queue. Decrements available copies and sends a BORROWED email notification.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| **book_id** | Long | Yes | ID of the book to borrow |
| **member_id** | Long | Yes | ID of the member borrowing the book |

### Example Request

```
GET /lms/borrowBook/?book_id=-3456789012345&member_id=-987654321
```

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Borrowed successfully | "Book borrowed successfully. Transaction ID: -112233\nPlease note this ID — it is required when returning the book." |
| **500** | Book not available | "Book 'Clean Code' is not available. Please reserve the book to join the reservation queue." |
| **500** | Borrow limit exceeded | "Borrowing limit exceeded. Current: 10, Limit: 10" |
| **500** | Member not found | "Member not found with ID: 999" |
| **200** | Circuit breaker open (fallback) | "The borrowing service is temporarily unavailable. Please try again in a few moments." |

⚠ Store the Transaction ID from the response — it is required to return the book via processReturn.

## 5.3 GET /lms/processReturn/{transactionId}

Returns a borrowed book. Calculates fines for overdue returns. If there are queued reservations for the book, the first one is activated and a RESERVATION_AVAILABLE notification is sent to that member.

### Path Parameter

| Field | Type | Required | Description |
|---|---|---|---|
| **transactionId** | Long | Yes | The transaction ID returned by borrowBook |

### Example Request

```
GET /lms/processReturn/-112233
```

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Returned on time | "Book 'Clean Code' returned successfully ON TIME. Thank you for returning by the due date (2024-02-14T10:30)!" |
| **200** | Returned late (fine) | "Book 'Clean Code' returned LATE. Overdue fine: $3.25 (13.0 days late). Due date was: 2024-02-14T10:30. Returned on: 2024-02-27T09:15" |
| **500** | Transaction not found | "Transaction not found with ID: 999" |
| **500** | Already returned | "Transaction #112233 is already closed. Status: RETURNED" |

## 5.4 GET /lms/getAllTransactions

Returns all transaction records in the system.

### Response — 200 OK (abbreviated)

```
[
  {
    "id":                -112233,
    "borrowDate":        "2024-01-15T10:30:00",
    "returnedDate":      null,
    "transactionStatus": "ACTIVE",
    "book": {
      "id":    -3456789012345,
      "title": "Clean Code",
      ...
    }
  }
]
```

# 6. Reservation Endpoints

Reserve a book that is currently unavailable. Reservations use a queue system — new reservations start as QUEUED, then move to ACTIVE (2-day pickup window) when the book is returned.

## 6.1 Endpoint Summary

| Method | Endpoint | Description |
|---|---|---|
| POST | /lms/reserveBook | Reserve a book for a member |
| POST | /lms/processReservationPickup/{reservationId} | Mark a queued reservation as picked up |
| POST | /lms/expireActiveReservations | Expire all overdue active reservations (cron/admin) |

## 6.2 POST /lms/reserveBook

Adds the member to the reservation queue for a book that is currently unavailable. Validates that: the book is actually unavailable, the member has no existing reservation for it, and the member has not exceeded their reservation limit.

### Query Parameters

| Field | Type | Required | Description |
|---|---|---|---|
| **book_id** | Long | Yes | ID of the book to reserve |
| **member_id** | Long | Yes | ID of the member making the reservation |

### Example Request

```
POST /lms/reserveBook?book_id=-3456789012345&member_id=-987654321
```

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Reservation created | "Reservation created with id:-567890\nYour reservation queue number is:2" |
| **500** | Book is currently available | "Book 'Clean Code' is currently available. Please borrow it directly instead of reserving." |
| **500** | Already reserved | "You already have an active reservation for 'Clean Code'" |
| **500** | Reservation limit exceeded | "Reservation limit exceeded. Current active reservations: 2, Limit: 2" |

✓ A RESERVATION_CREATED email is sent automatically to the member upon success.

## 6.3 POST /lms/processReservationPickup/{reservationId}

Marks a reservation as FULFILLED when the member comes to collect the book. The reservation must be in ACTIVE status (not QUEUED or EXPIRED).

### Path Parameter

| Field | Type | Required | Description |
|---|---|---|---|
| **reservationId** | Long | Yes | The reservation ID returned when the reservation was created |

### Responses

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Pickup successful | "Reservation fulfilled. Book borrowing successfull." |
| **500** | Book not available | "Book 'Clean Code' is not available. Status: Unavailable" |
| **500** | Reservation not found | "Reservation not found with ID: 999" |
| **500** | Wrong status (not ACTIVE) | "Can only fulfill ACTIVE reservations. Current status: QUEUED" |

## 6.4 POST /lms/expireActiveReservations

Scans all ACTIVE reservations and marks any that have passed their 2-day expiry window as EXPIRED. Intended to be called by a scheduled job or admin action — not typically triggered by the user interface.

### Request

No body or parameters.

### Response

| Status | Meaning | Response Body / Notes |
|---|---|---|
| **200** | Completed (no body) | Empty response body — HTTP 200 is the success indicator |

# 7. Resilience & Circuit Breaker

Two circuit breakers guard the core book and member service operations. When a circuit opens, fallback responses are returned immediately instead of waiting for a timeout.

## 7.1 GET /lms/circuitBreakers

Returns the current state of all registered circuit breakers. Useful for debugging and health dashboards.

### Response — 200 OK

```
{
  "bookService": {
    "state":                    "CLOSED",
    "failureRate":              "0.0%",
    "numberOfBufferedCalls":    5,
    "numberOfFailedCalls":      0,
    "numberOfSuccessfulCalls":  5,
    "description":              "Normal operation - requests passing through"
  },
  "memberService": { ... }
}
```

## 7.2 Circuit Breaker States

| Value | Description |
|---|---|
| **CLOSED** | Normal operation — all requests pass through. |
| **OPEN** | Too many failures detected. Requests are blocked; fallback is returned immediately. |
| **HALF_OPEN** | Recovery mode — a limited number of trial requests are allowed to test if service recovered. |

## 7.3 Dev vs Prod Thresholds

| Setting | Dev | Prod | Notes |
|---|---|---|---|
| failure-rate-threshold | 80% | 50% | % of calls that must fail to open circuit |
| wait-duration-in-open-state | 5s | 30s | How long circuit stays open before trying |
| sliding-window-size | N/A | 10 calls | Number of recent calls evaluated |

# 8. Notification Flow

Notifications are sent automatically by the LMS Core service after key operations. The frontend does not call notification endpoints directly.

| Trigger | Notification Type | Email Subject Sent |
|---|---|---|
| borrowBook success | BORROWED | Book Borrowed Successfully |
| processReturn (on time) | RETURNED | Book Returned |
| processReturn (overdue) | OVERDUE | Overdue Notice - Fine Applied |
| reserveBook success | RESERVATION_CREATED | Reservation Confirmed |
| processReturn activates next reserved | RESERVATION_AVAILABLE | Your Reserved Book is Available! |

**i** Notification failures do not break borrowing or return flows. A Resilience4j retry (3 attempts, 2s wait) is applied to all notification publishes.

# 9. Common Frontend Workflows

## 9.1 Register Member and Borrow a Book

Step-by-step flow for the most common user journey:

- POST /lms/registerMember → Save the member ID from the response string
- POST /lms/addBook → Save the book ID from the response string
- GET /lms/borrowBook/?book_id=<id>&member_id=<id> → Save the transaction ID
- GET /lms/processReturn/<transactionId> → Complete the return

⚠ IDs returned by the API are large random Longs (e.g. -3456789012345). Parse them as strings or 64-bit integers — they will overflow 32-bit integers.

## 9.2 Reserve a Book (Unavailable)

- Confirm the book is unavailable: GET /lms/getAllBooks → find book with copiesAvailable = 0
- POST /lms/reserveBook?book_id=<id>&member_id=<id> → Save reservation ID
- When notified (RESERVATION_AVAILABLE email), call POST /lms/processReservationPickup/<id>

## 9.3 Check Circuit Breaker Health

- GET /lms/circuitBreakers → Check that bookService and memberService states are CLOSED
- If state = OPEN, display a user-facing message: "Service temporarily unavailable, try again shortly"

# 10. Error Handling Guide for Frontend

| HTTP | Scenario | Recommended Frontend Action |
|---|---|---|
| 200 | Success | Parse response string for IDs if needed. Most responses are plain text. |
| 429 | Rate limit exceeded | Show: "Too many requests. Please wait a moment and try again." |
| 500 | Business rule violation | Read the message field from the error JSON and display it to the user. Most 500s here are domain errors (not available, limit exceeded), not server crashes. |
| 500 | Entity not found | Display: "Item not found. Please refresh and try again." |

⚠ Note: The backend uses HTTP 500 for both business validation errors and unexpected server errors. Read the message field to distinguish them.

## 10.1 Parsing IDs from Plain Text Responses

Several endpoints return plain-text strings with embedded IDs. Example patterns:

```
"Book added with Id: -3456789012345"
```

Suggested regex for extracting ID: /-?\d+/

```
"Reservation created with id:-567890\nYour reservation queue number is:2"
```

Split on newline, then parse the integer after the colon in each line.

# 11. Service Startup Order

Services must be started in the following order for the system to function correctly:

| # | Service | Port | Wait For |
|---|---|---|---|
| 1 | **Eureka Server** | 8761 | Nothing — start first |
| 2 | **Config Server** | 8888 | Eureka to be up (registers with it) |
| 3 | **Notification Service** | 8085 | Config Server to serve notificationservice.yml |
| 4 | **LMS Core** | 8081 | All above services to be registered in Eureka |

ℹ LMS Core uses load-balanced RestTemplate with Eureka. If Notification Service is not registered, notification calls will fail (but borrowing/returning will still succeed due to the retry + fallback design).

*End of Document — LMS Backend API Reference v1.0*